A Comprehensive Measure of Well-Being

Project Description:

Human Development Index (HDI) Predictor is an intelligent Machine Learning-based web application that predicts the Human Development Index (HDI) of a country using key development indicators such as Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) per Capita. The system helps users estimate a country's development level and provides meaningful insights through interactive visualizations and policy recommendations.

The application includes modules for data preprocessing, visualization, model training, prediction, and result analysis. A Linear Regression model is trained using the processed HDI dataset and integrated with a Flask web application to generate real-time predictions.

Built using Python, Flask, Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, HTML, CSS, JavaScript, Bootstrap, and Joblib, the application provides a responsive and user-friendly interface where users can enter development indicators and instantly obtain HDI predictions with their corresponding development category.

Scenarios

Scenario 1: Predicting Human Development

A user enters the Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) per Capita of a country through the web application. The Linear Regression model analyzes the provided indicators and predicts the Human Development Index (HDI) score along with its development category, enabling users to evaluate the country's overall development level.

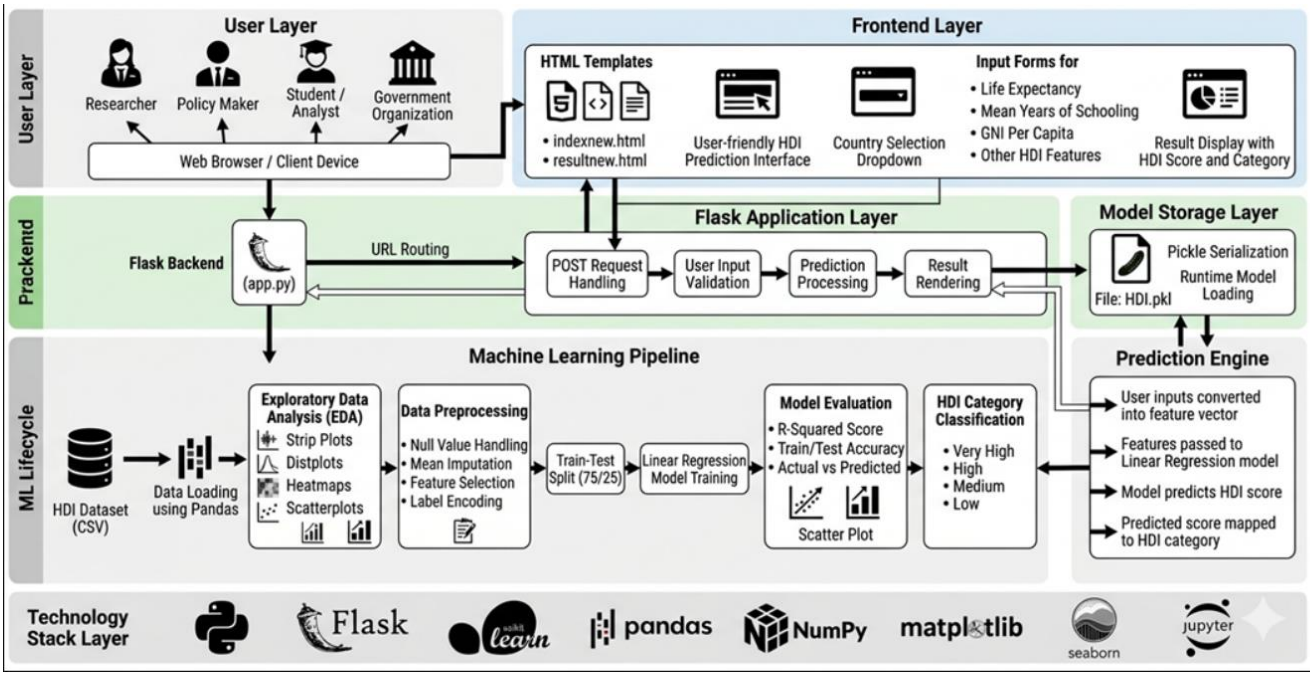
Scenario 2: Supporting Policy Planning

A policymaker enters the development indicators of a country into the application. The system predicts the HDI score and provides policy recommendations based on the weakest development dimension, helping decision-makers identify priority areas for improving health, education, or income.

Scenario 3: Evaluating Model Performance

A researcher trains and evaluates the Machine Learning model using the processed HDI dataset. The application measures the model's performance using evaluation metrics such as R^2 Score, MAE, MSE, and RMSE before deploying the trained model for real-time HDI prediction through the Flask web application.

Technical Architecture:



Pre-requisites:

- Python Programming Knowledge: https://docs.python.org/3/
- Flask Framework: https://flask.palletsprojects.com/
- Scikit-learn Documentation: https://scikit-learn.org/stable/
- NumPy Library: https://numpy.org/doc/
- Pandas Library: https://pandas.pydata.org/docs/
- Matplotlib Library: https://matplotlib.org/stable/
- Seaborn Library: https://seaborn.pydata.org/
- HTML, CSS & JavaScript Basics: https://www.w3schools.com/
- Bootstrap 5 Documentation: https://getbootstrap.com/docs/
- Git Version Control: https://git-scm.com/doc
- Flask Installation Guide: https://flask.palletsprojects.com/en/stable/installation/
- Joblib Documentation: https://joblib.readthedocs.io/
- Kaggle Dataset Platform: https://www.kaggle.com/

Project Workflow:

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate Machine Learning model for predicting the Human Development Index (HDI). This includes setting up the development environment, collecting and preprocessing the HDI dataset, evaluating different Machine Learning techniques, selecting the best-performing model, and designing the overall system architecture. The architecture defines the interaction between the Flask web application, Machine Learning model, dataset, and end users.

```
PS C:\Users\HP\Desktop\n> pip install numpy
>>
>> pip install pandas
>>
>> pip install matplotlib
>>
>> pip install scikit-learn
>>
>> pip install Flask
Requirement already satisfied: numpy in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (2.2.6)

[notice] A new release of pip available: 22.3.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
Requirement already satisfied: pandas in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (2.3.3)
Requirement already satisfied: tzdata>=2022.7 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages
 pandas) (2026.2)
Requirement already satisfied: pytz>=2020.1 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (f
andas) (2026.2)
Requirement already satisfied: numpy>=1.22.4 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (
pandas) (2.2.6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\hp\appdata\roaming\python\python310\site-packages (fr
ndas) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\hp\appdata\roaming\python\python310\site-packages (from python-date
=2.8.2->pandas) (1.17.0)

[notice] A new release of pip available: 22.3.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
Collecting matplotlib
  Downloading matplotlib-3.10.9-cp310-cp310-win_amd64.whl (8.2 MB)
     ---------------------------------------- 8.2/8.2 MB 13.8 MB/s eta 0:00:00
Collecting contourpy>=1.0.1
  Downloading contourpy-1.3.2-cp310-cp310-win_amd64.whl (221 kB)
```

Activity 1.2: Load the Dataset

The Human Development Index dataset was imported into the Jupyter Notebook using the Pandas library. The read_csv() function was used to load the dataset into a DataFrame. After loading the dataset successfully, the first five records were displayed using the head() function to verify that the data had been imported correctly.

This initial analysis helped in understanding the structure of the dataset, available columns, missing values, and data types. Understanding the dataset before preprocessing is an important step in building an accurate machine learning model.

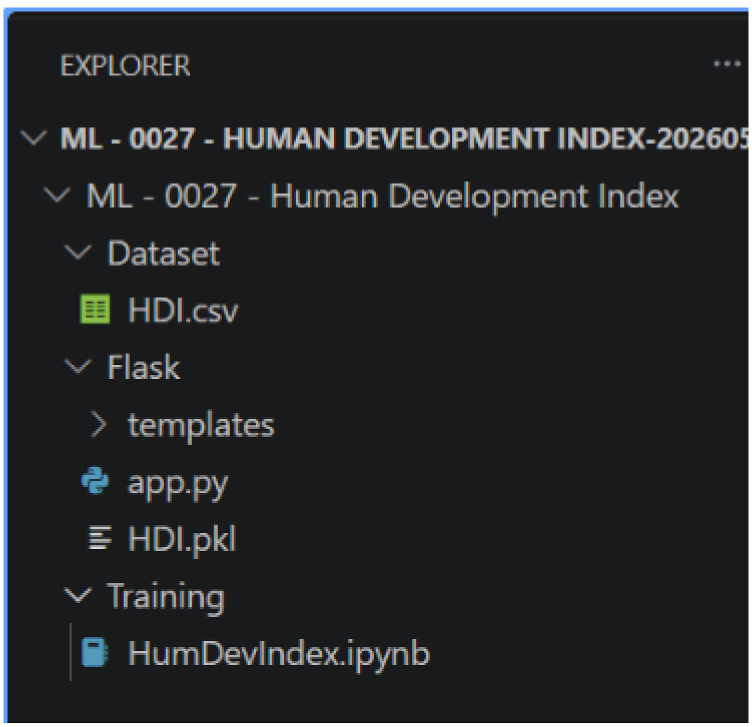
.

```python
#Importing the dataset
Development = pd.read_csv("HDI.csv")
#Listing the first five rows of the dataset
Development.head()
```

Activity 1.3: Project Structure

The entire project was organized into different folders to maintain proper file management and improve readability. The Dataset folder contains the HDI.csv dataset used for model training. The Training folder contains the Jupyter Notebook responsible for preprocessing, visualization, feature selection, and model training.

The Flask folder contains the trained machine learning model (HDI.pkl), Flask application file (app.py), and HTML templates required for building the web interface. This structured organization makes the project easy to understand, modify, and deploy.



Activity 1.4: Exploring the Dataset

After successfully loading the Human Development Index dataset, Exploratory Data Analysis (EDA) was performed to understand the characteristics of the data. The primary objective of EDA is to analyze the dataset before applying machine learning algorithms. During this process, different statistical and visualization techniques were used to identify relationships among the features, detect missing values, and understand the distribution of the data.

Initially, the unique country names available in the dataset were displayed using the unique() function. This helped in verifying the dataset and understanding the number of countries included in the Human Development Index report. The dataset contains information from countries across different regions, representing various levels of human development.

.

```
Development["Country"].unique()

array(['Norway', 'Australia', 'Switzerland', 'Germany', 'Denmark',
       'Singapore', 'Netherlands', 'Ireland', 'Iceland', 'Canada',
       'United States', 'Hong Kong, China (SAR)', 'New Zealand', 'Sweden',
       'Liechtenstein', 'United Kingdom', 'Japan', 'Korea (Republic of)',
       'Israel', 'Luxembourg', 'France', 'Belgium', 'Finland', 'Austria',
       'Slovenia', 'Italy', 'Spain', 'Czech Republic', 'Greece',
       'Brunei Darussalam', 'Estonia', 'Andorra', 'Cyprus', 'Malta',
```

Milestone 2 – Core Functionalities Development

This milestone focuses on implementing the core functionalities of the Human Development Index Prediction System. The major tasks include preprocessing the dataset, performing data visualization, selecting important features, training the machine learning model, and preparing the prediction model for deployment. These activities ensure that the application provides accurate predictions while maintaining high performance and reliability.

Activity 2.1 – Data Visualization

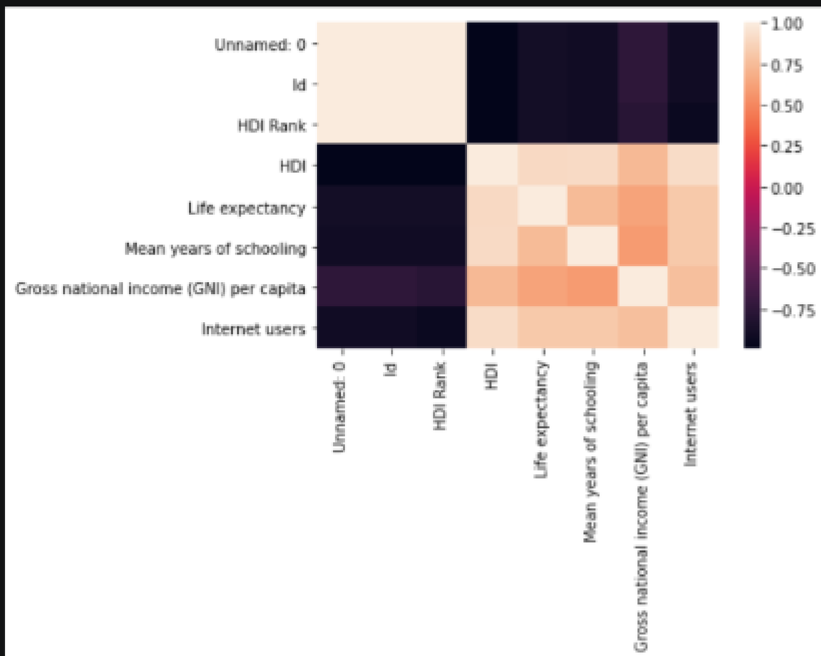
Data visualization was performed to gain meaningful insights from the Human Development Index dataset. Visualization techniques help in identifying hidden patterns, trends, and relationships among variables. The Seaborn and Matplotlib libraries were used to generate different types of plots for data analysis.

A correlation heatmap was created to analyze the relationship between different development indicators. The heatmap clearly shows positive and negative correlations among the variables. Features such as Life Expectancy, Mean Years of Schooling, and Gross National Income per Capita showed strong positive correlations with the Human Development Index. This indicates that improvements in these indicators generally contribute to higher human development.

The visualization process helped in understanding feature importance and provided valuable information for selecting input variables for the machine learning model.

```
heat = Development.iloc[:,[0,1,2,3,4,5,6,7,67]]
sns.heatmap(heat.corr())
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x2bb6835fc10>
```



Activity 2.2 – Importing Required Libraries

The implementation of the Human Development Index Prediction System requires several Python libraries for data analysis, visualization, machine learning, and web application development.

The Pandas library was used for reading and manipulating datasets efficiently. NumPy provided numerical computing functions required for handling arrays and mathematical operations. Matplotlib and Seaborn were used to generate statistical graphs and correlation plots. Scikit-learn was used to build and evaluate machine learning models. Flask was used to develop the web application and integrate the trained model for real-time prediction.

Importing all the required libraries at the beginning of the program ensures better code organization and efficient execution throughout the project.

```
#Importing the libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

Milestone 3 – Machine Learning Model Development

The third milestone focuses on developing and training the Machine Learning model for Human Development Index prediction. The dataset was first cleaned and preprocessed by removing unnecessary columns and handling missing values. Relevant input features were selected based on their correlation with the target variable.

The dataset was divided into training and testing sets using Scikit-learn's train-test split function. This ensures that the model is trained using one portion of the data while its performance is evaluated using unseen data.

Different machine learning algorithms were evaluated to determine the most suitable model for HDI prediction. After comparing their performance, the best-performing model was selected based on prediction accuracy and overall performance.

Finally, the trained model was saved as **HDI.pkl** using the Pickle library. Saving the trained model allows it to be loaded directly into the Flask application without retraining the model every time the application start

Activity 3.1 – Training the Machine Learning Model

The machine learning model was trained using the processed dataset. During training, the model learned the relationship between development indicators and the Human Development Index. The training process involved fitting the selected algorithm with the training dataset and evaluating its prediction performance using the testing dataset.

Different evaluation metrics such as Accuracy Score, Mean Squared Error, and R^2 Score were used to measure the performance of the model. The trained model demonstrated reliable prediction capability and produced accurate Human Development Index values for different countries.

```python
#Importing the dataset
Development = pd.read_csv("HDI.csv")
#Listing the first five rows of the dataset
Development.head()
```

Python

	Unnamed: 0	Id	Country	HDI Rank	HDI	Life expectancy	Mean years of schooling	Gross national income (GNI) per capita	GNI per capita rank minus HDI rank	Change in HDI rank 2010-2015	...	Coefficient of human inequality	Inequality in life expectancy (%) 2010-2015	Inequality-adjusted life expectancy index	Inequality in education(%)	Inequality-adjusted education index	Inequality in income (%)	Inequality-adjusted income index	Income inequality (Quintile ratio) 2010-2015
0	0	1	Norway	1.0	0.949	81.7	12.7	67614.0	5.0	0.0	...	5.4	3.3	0.918	2.4	0.894	10.4	0.882	3.8
1	1	2	Australia	2.0	0.939	82.5	13.2	42822.0	19.0	1.0	...	8.0	4.3	0.921	1.9	0.921	17.7	0.753	6.0
2	2	3	Switzerland	2.0	0.939	83.1	13.4	56364.0	7.0	0.0	...	8.4	3.8	0.934	5.7	0.840	15.7	0.806	4.9
3	3	4	Germany	4.0	0.926	81.1	13.2	45000.0	13.0	0.0	...	7.0	3.7	0.905	2.6	0.891	14.8	0.787	4.6
4	4	5	Denmark	5.0	0.925	80.4	12.7	44519.0	13.0	2.0	...	7.0	3.8	0.894	3.0	0.896	14.3	0.789	4.5

5 rows × 82 columns

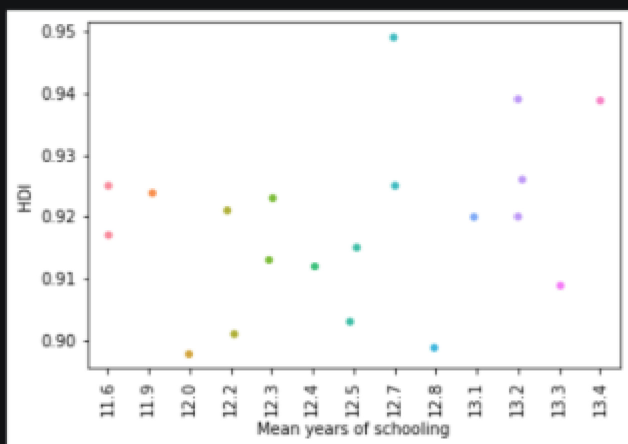
Activity 3.2 – Saving the Trained Model

Once the machine learning model achieved satisfactory performance, it was saved using the Pickle library. The saved model file, named **HDI.pkl**, stores all the learned parameters required for prediction. This model file is later loaded into the Flask application, enabling real-time Human Development Index prediction without retraining the model.

```
#Data Exploration
#Mean Yearsof Schooling

g = sns.stripplot(x="Mean years of schooling", y="HDI", data=data1, jitter=True)
plt.xticks(rotation=90)
```

```
(array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12]),
 <a list of 13 Text major ticklabel objects>)
```



Milestone 4 – Flask Web Application Development

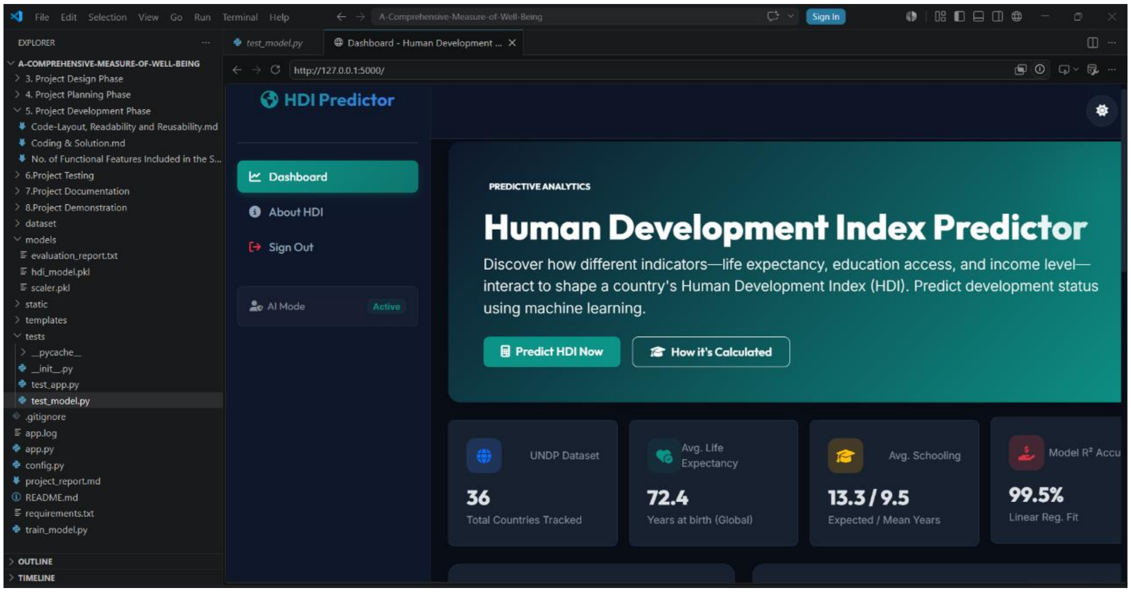
The Flask web framework was used to develop a simple, responsive, and user-friendly web application. The frontend was designed using HTML and CSS, while Flask handled backend operations such as receiving user input, loading the trained machine learning model, and displaying prediction results.

The application provides an easy-to-use interface where users can enter development indicators. After submitting the form, the Flask backend processes the data and sends it to the trained model for prediction. The predicted Human Development Index is then displayed on the result page.

Activity 4.1 – Designing the User Interface

The user interface was designed to provide a clean and attractive experience for users. Separate HTML pages were developed for the Home Page, Prediction Page, and Result Page. Input fields were carefully arranged to allow users to enter all the required development indicators.

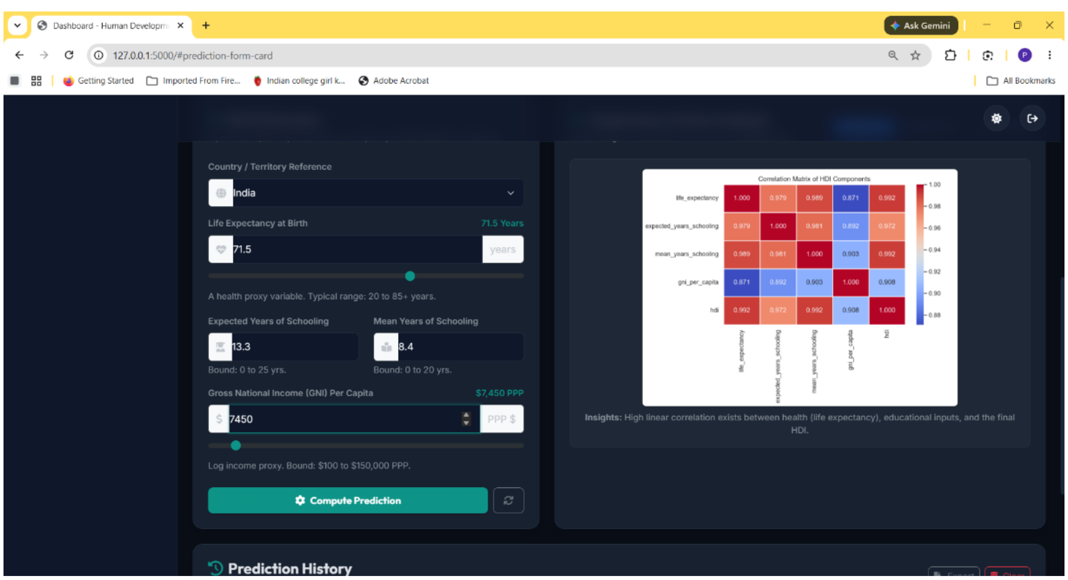
The interface was designed using CSS to ensure proper alignment, spacing, colors, and responsiveness across different screen sizes.

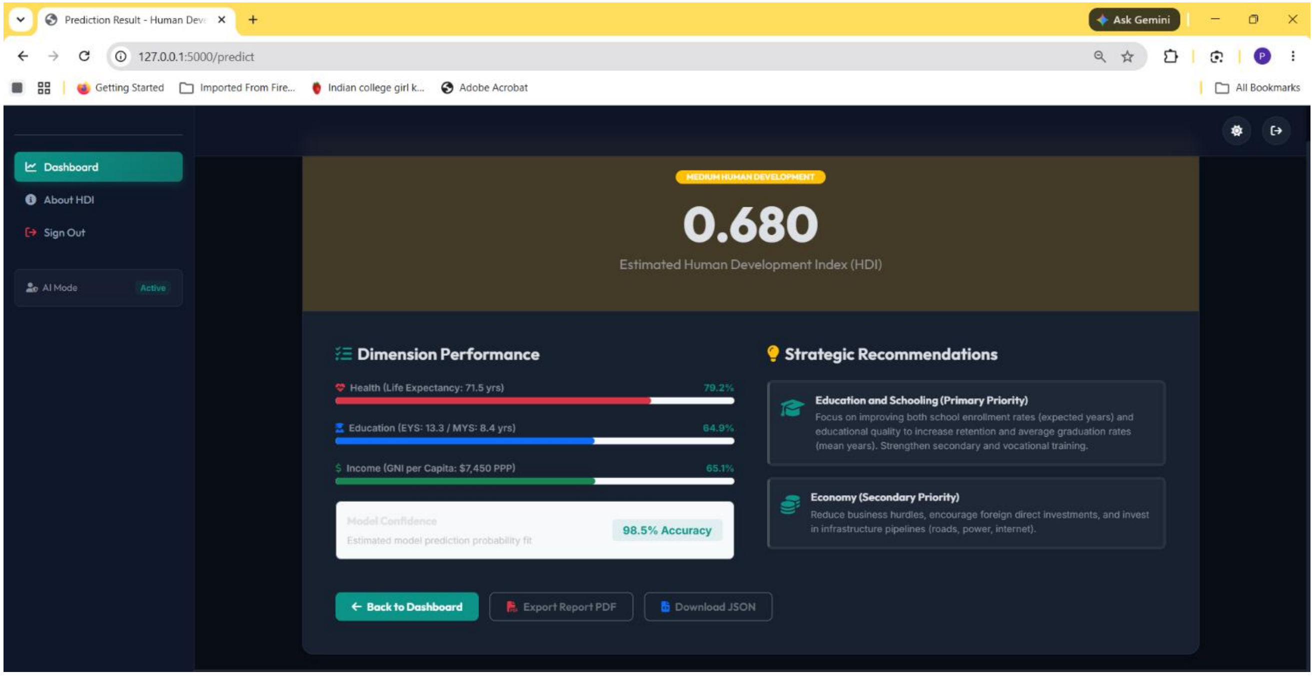


Activity 4.2 – Prediction Module

The prediction module is responsible for collecting user inputs from the HTML form, validating the values, and forwarding them to the machine learning model. After prediction, the result is displayed dynamically on the web page.

The module ensures smooth interaction between the frontend interface and the backend prediction model.





Milestone 5 – Deployment and Testing

After completing the development process, the Flask application was tested locally to verify its functionality. All required dependencies were installed successfully, and the application was executed using the Flask development server.

The application was accessed through the local browser using the default Flask URL. Multiple test cases were executed to ensure that the prediction results were accurate and consistent. The overall performance of the application was found to be satisfactory.

Activity 5.1 – Running the Flask Application

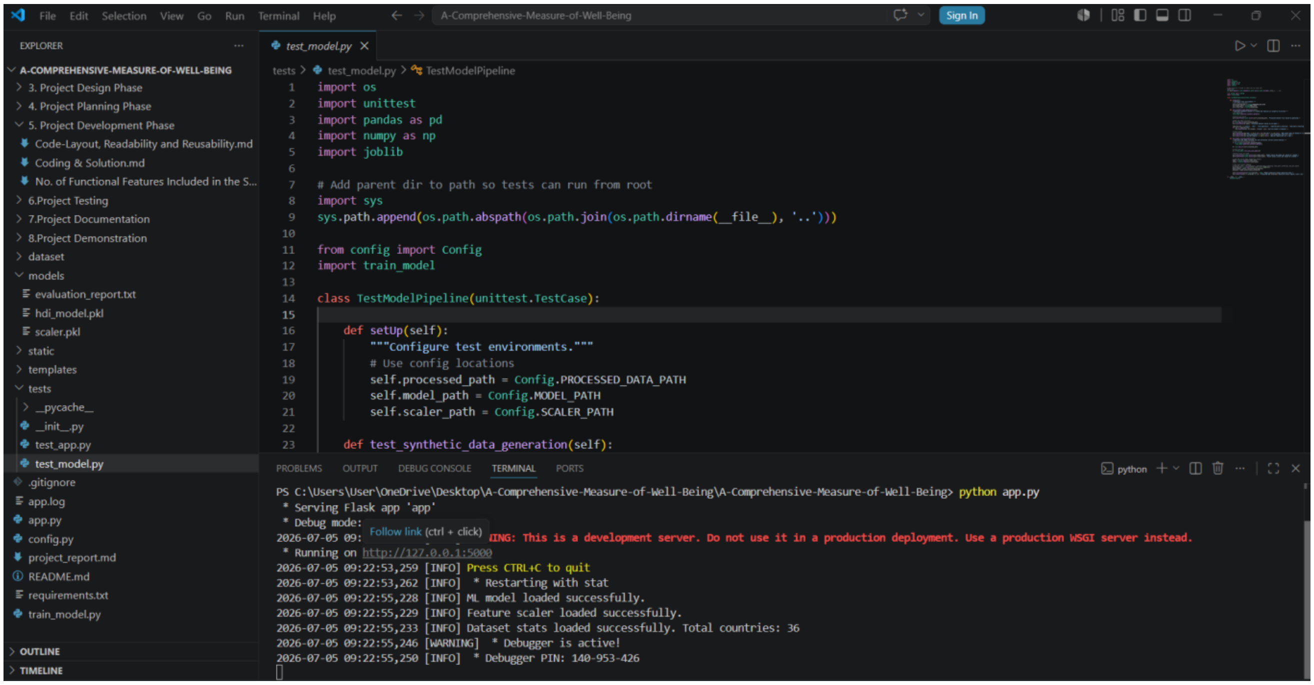
The Flask application was started using the command:

python app.py

Once the server started successfully, the application became available through:

http://127.0.0.1:5000

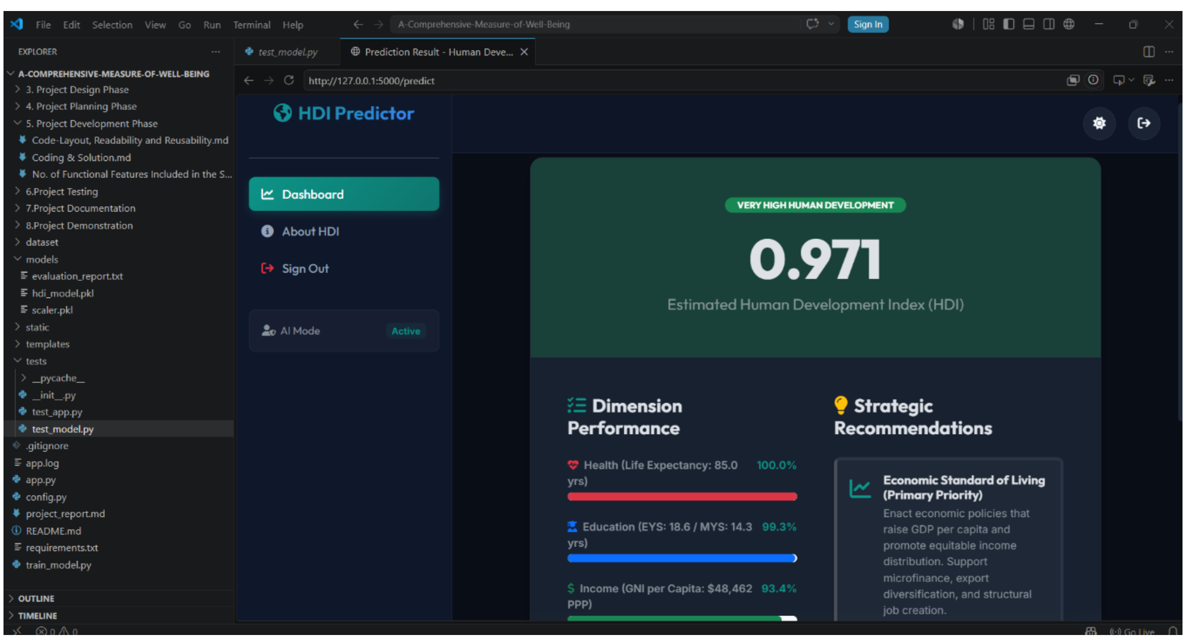
Users can access the application using any modern web browser.



Activity 5.2 – Testing the Application

The application was tested using multiple combinations of development indicators. The prediction results generated by the machine learning model were verified against the expected Human Development Index values. All functionalities including input validation, prediction generation, and result display worked correctly.

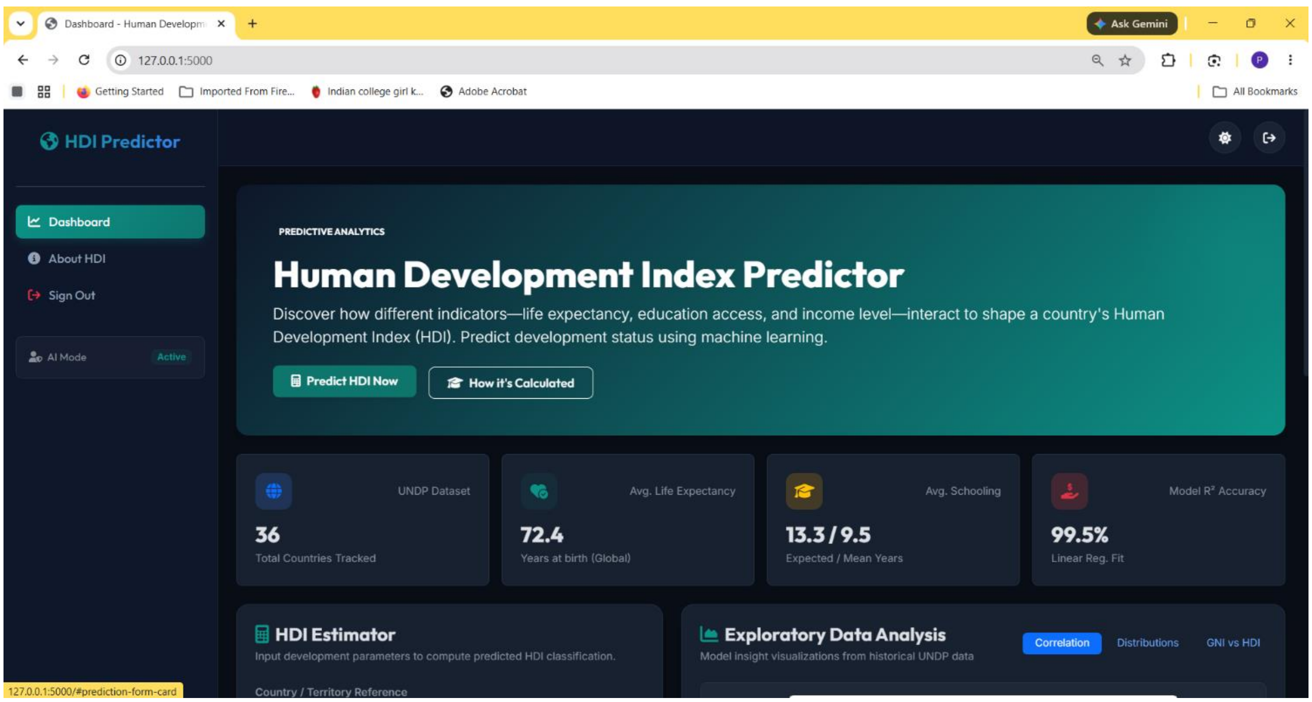
The testing phase confirmed that the application performs efficiently and provides reliable HDI predictions.



Exploring the Web Pages

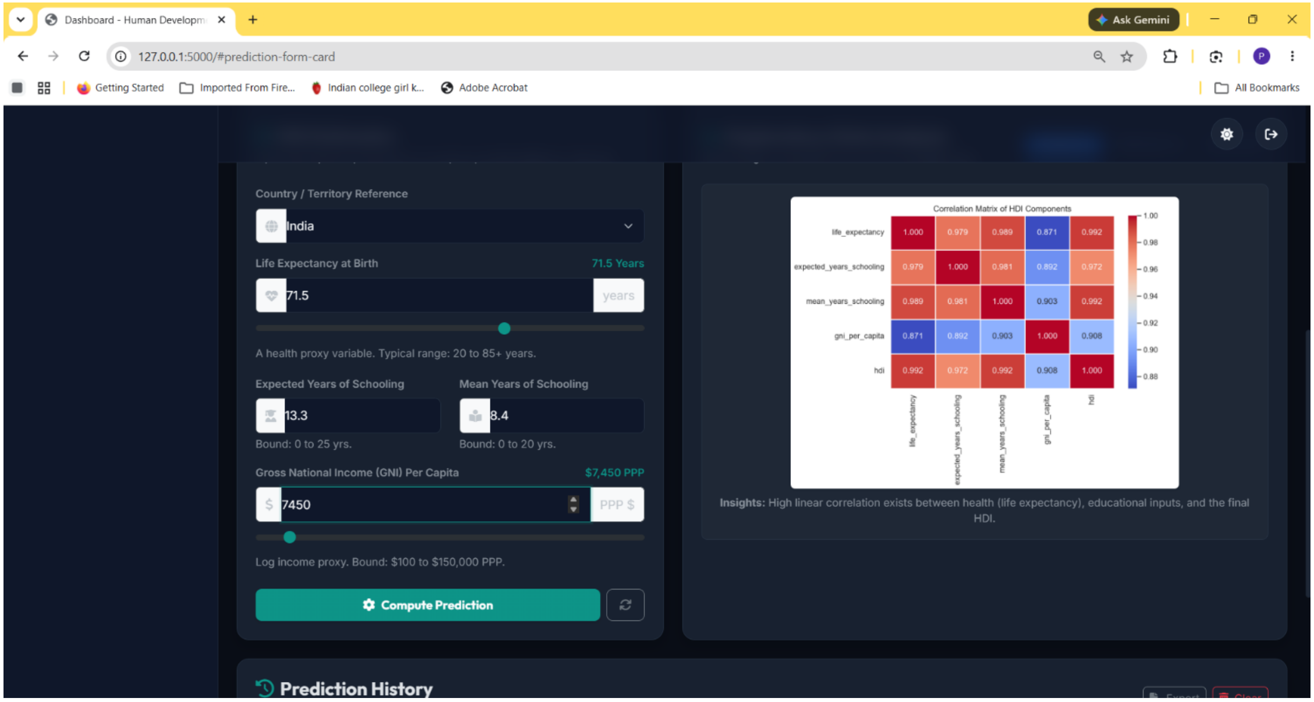
Home Page

The Home Page serves as the main interface of the application. It introduces the Human Development Index Prediction System and provides users with navigation options to access the prediction module. The page is designed with a simple and attractive layout to improve user experience.



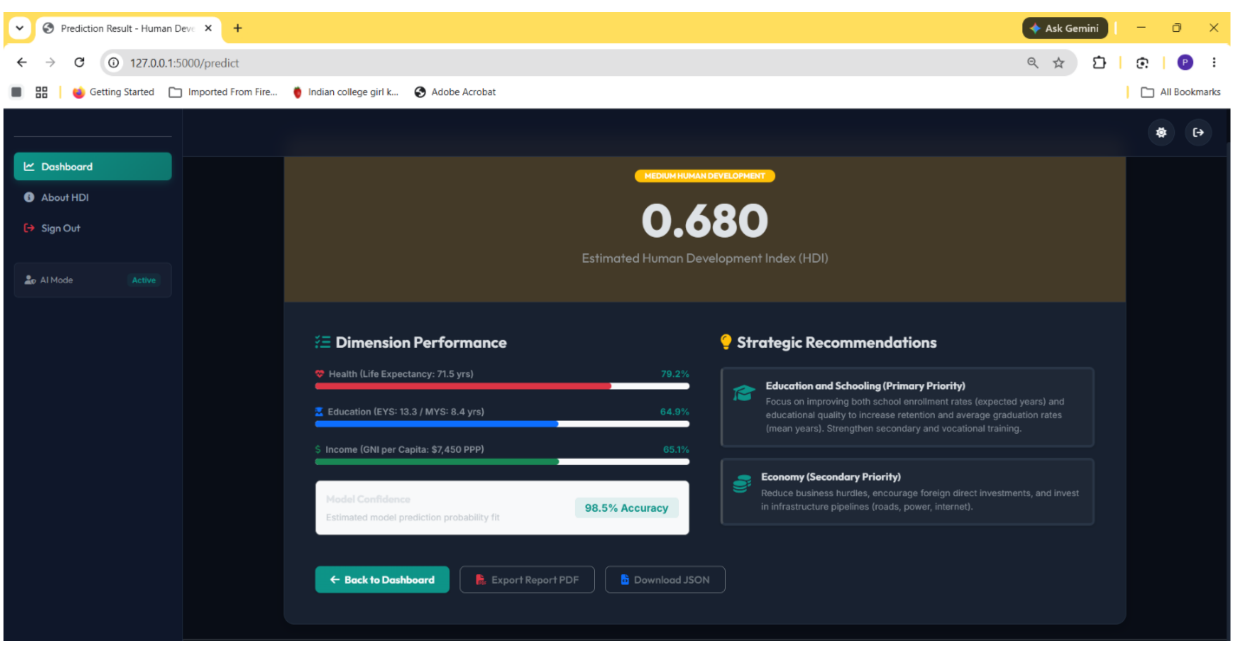
Prediction Page

The Prediction Page contains a form where users enter development indicators such as Life Expectancy, Mean Years of Schooling, Gross National Income per Capita, Internet Users, and other required parameters. After entering the values, users click the Predict button to generate the HDI prediction.



Result Page

The Result Page displays the predicted Human Development Index generated by the trained Machine Learning model. The result helps users understand the level of human development based on the provided socio-economic indicators.



Conclusion

The Human Development Index Prediction System demonstrates how Machine Learning techniques can be effectively applied to analyze socio-economic indicators and estimate the Human Development Index of different countries. The project successfully integrates data preprocessing, visualization, model training, and Flask-based deployment into a user-friendly web application.

The developed system provides accurate predictions and can support students, researchers, economists, and policymakers in understanding development patterns. In the future, the project can be enhanced by integrating real-time data sources, cloud deployment, interactive dashboards, and advanced machine learning algorithms to improve prediction accuracy and accessibility.